Contents

1 Introduction **2**
1.1 Motivation 2

2 Theoretical Background **4**
2.1 Technical Substrate 4
2.2 Related Work: Natural Language to GIS 6
2.2.1 The Autonomous GIS Vision 6
2.2.2 Grounded, Tool-Augmented Assistants 6
2.2.3 Benchmarking and Evaluation Practice 7
2.2.4 Multi-Agent and Hybrid Architectures 7
2.3 Operational Assistants and Their Design Choices 7

Chapter 1

Introduction

1.1 Motivation

Geographic information systems (GIS) have become the standard instrument for working with spatial data at every scale of public administration, science and industry. The reason for their success is not only analytical power but *reproducibility*: an analysis performed in a desktop application can be recorded as a script and re-executed on new data, on another machine, by another analyst. For a long time this reproducibility was obtained from the graphical user interfaces of systems such as GRASS (14) and QGIS (17), whose Python bindings (16, 7) let an analyst convert a click sequence into a re-runnable script after the fact.

That order of work, however, is inverted with respect to how analytical needs are actually formulated. A practitioner usually knows the analysis in *intent* long before knowing its implementation: it is clear that the streams must be buffered by 200 metres, or that average temperature must be computed per administrative unit, while the exact module name, the spelling of the parameters and the calling convention remain an implementation detail to be looked up. The gap between intent and implementation is small in knowledge but large in friction, and it is paid for on every single analysis.

Large language models (LLMs) appear to dissolve that gap: a model can be asked for an analysis in ordinary language and can answer with code (22, 2). Since 2023 this capability has been repackaged into *agents*: systems in which a language model selects tools, executes them, observes the outcome and iterates (25). The geospatial community has adopted this direction vigorously, both through research prototypes (28, 29, 21) and through a rapidly growing ecosystem of QGIS plugins that put a chat window in front of a running desktop session.

The enthusiasm, however, meets a hard constraint. The most capable code models available in practice are large, and the most convenient way of running them is as a hosted service. For an analyst handling sensitive spatial data — land cadastre, utility networks, health data, defence-related imagery — neither the upload of the prompt, nor the projection of the resulting code into a vendor's cloud, nor the loss of control over which model version produced a given analysis are acceptable. A large part of the current literature on LLM-assisted GIS silently assumes a cloud endpoint (12, 21). An offline alternative must therefore

work with what a workstation can run: a model of a few billion parameters, a local inference server, and the documentation that ships with the software being automated. Offline-first operation and hybrid routing are not mutually exclusive: keeping sensitive requests local while allowing non-sensitive ones to reach a stronger hosted model gives the analyst both control and capability.

This thesis addresses that gap. It develops and evaluates GrassAgent, an offline-first agent that translates unconstrained natural-language GIS requests into executable Python scripts written against the modern object-oriented `grass.tools` API of GRASS, verifies those scripts deterministically against the official GRASS manual, and — as a second stage — executes them in a live headless GRASS session and attempts to repair them from the resulting error feedback.

Chapter 2

Theoretical Background

This chapter establishes the conceptual and technical vocabulary on which the remainder of the thesis depends, and reviews the literature into which the presented system is placed. Section 2.1 condenses the technical substrate: the GIS environment that is automated, the language-model machinery that performs the translation, the retrieval and routing layers, and the program-analysis techniques that supply the guarantees. Section 2.2 reviews the prior art, beginning with the method of the search, and Section 2.3 examines the operational assistants most closely comparable to the presented system.

2.1 Technical Substrate

The GIS substrate. GRASS (14) is an open-source GIS whose 8.6 release exposes more than five hundred and forty modules spanning raster, vector, imagery, database, display and 3D processing; an agent choosing among them is solving a selection problem, not a memorisation problem. It is dual-interface: every module is reachable from the command line and, since version 8.5, through a machine-generated Python wrapper in which `r.slope.aspect` becomes `r_slope_aspect` and every parameter becomes a keyword argument (7). The two interfaces are functionally equivalent but textually dissimilar, so a model pretrained mostly on the command line is systematically biased towards the wrong surface syntax. Execution is also unusually clean — analysis operates inside a *location* containing *mapsets*, and every operation is a module invocation with named options — which makes process-level isolation of generated scripts straightforward. The data model distinguishes raster and vector data, with attribute tables attached to vector features; the distinction determines which modules may consume which input and is a recurring source of ambiguity in requests that mix "each cell" with "points". Add-ons installed with `g.extension` register alongside core modules and are reachable through the same interfaces. QGIS (17), by contrast, exposes Python bindings (16) and a Processing framework of several hundred algorithms invoked with a uniform identifier and parameter dictionary; that uniform signature explains why much academic work targets PyQGIS (12), whereas GRASS is comparatively under-represented and its dual interface is a sharper test of grounding.

Language models for code. Current code-capable models rest on the transformer architecture (22), and scaling it produced general-purpose models with emergent code-completion ability (1). Code-specialised models trained on function–documentation pairs outperform general models of the same size on API-shaped generation (2), and code-specialised small models such as the Qwen2.5-Coder family (18) make offline deployment realistic rather than merely desirable. On a niche API these models hallucinate identifiers (26, 8) through three interacting mechanisms: *surface-form dominance*, since the older and more frequent command-line form dominates the prior over "how a GRASS operation looks in Python"; *compositional pressure*, which tempts the model to append a plausible but invented finalisation step; and *argument-position convention transfer*, since the command line names outputs while the wrapper returns them. These are grounding deficiencies amenable to context and verification, not reasoning deficiencies that a larger model alone would fix.

Retrieval-augmented generation. RAG (10) conditions a parametric model on text fetched from a non-parametric store, addressing knowledge that is absent from the weights and weights that cannot be updated. For an API surface the store is small and curated, so retrieval is closer to *interface lookup* than to open-domain question answering, and precision matters more than coverage. Dense retrieval (9) is robust to paraphrase; lexical retrieval such as BM25 (19) is precise when vocabulary matches and brittle when it does not. On short GIS prompts dense similarity alone tends to promote generic terrain modules over the specific module named, because a short query embedding is dominated by the broad topic; hybrid retrieval combines both, and Reciprocal Rank Fusion (3) combines rankings by position so that incomparable score scales do not matter. Domain re-ranking adds priors invisible to a similarity model: *category weighting* towards raster or vector modules according to vocabulary, *wrapper preference* for modern wrappers such as `v.import` over the raw drivers they replace, and *anti-crowding*, which collapses module families so that siblings cannot crowd out the correct module. Retrieved context must also be optimised: generic parameters such as `flags`, `overwrite` and `verbose` cause *parameter bleed* when left in rendered signatures, and corrupt documentation examples are faithfully imitated, so deterministic filtering is preferable to instructing the model to be careful.

Local inference and hybrid routing. A workstation-class model trades capability for control: it fits in memory and is fast enough for interactive use, but is weaker than a hosted frontier model on unfamiliar APIs. Routing chooses between a local and a remote backend per request on grounds of data sensitivity, task difficulty, cost or latency, and local servers such as Ollama and llama.cpp expose an OpenAI-compatible interface so that the same client can address either. A verification-first design tolerates a weaker local model because the validator rejects malformed output regardless of its origin, while cloud routing is acceptable only when the prompt contains no data that may not leave the host.

Verification and program analysis. Static verification over the Python abstract syntax tree (15) checks each generated call against a schema for module existence, required parameters,

unknown parameters, the keyword-only calling convention, and dictionary unpacking, which has no name in the tree and would otherwise escape the check. It is exact with respect to its schema and reproducible — no sampling, no temperature, no model in the loop — but says nothing about semantic correctness or whether input layers exist. The complementary technique executes the artifact and returns the observation as evidence. Automated program repair either patches a failure location (4) or re-prompts with the failing program and its output, and Reflexion (20) established the general form of the latter as verbal reinforcement, placing the model's own failure observation back into context without weight updates. Runtime feedback catches failures that static analysis cannot see — a missing layer, a type error, a wrong column name — at the risk of unbounded cost and non-convergence, which calls for an attempt ceiling and recorded failure modes.

2.2 Related Work: Natural Language to GIS

The review below was conducted as a structured search rather than as an accumulation of reading. Works were included if they translate natural language into geospatial code or workflows, evaluate language models on geospatial code, present an operational language-model-driven GIS assistant, or supply the generic machinery — retrieval, agents, program analysis — on which such a system depends; purely conversational assistants and remote-sensing vision-language work were excluded as non-comparable. Searches covered the academic indexes and preprint servers, supplemented by citation chasing and by a systematic survey of the official QGIS plugin repository. Because the field and the plugin ecosystem move quickly, comparative statements reflect the state at the date of access.

2.2.1 The Autonomous GIS Vision

The earliest substantial statement of the problem addressed here is the *Autonomous GIS* proposal, in which a language model acts as the reasoning engine of a GIS, decomposing a task into sub-tasks and generating code for each (11). The contribution was primarily conceptual: it established that the interesting target is not a single function call but a workflow, and that the bottleneck is task decomposition rather than syntax.

2.2.2 Grounded, Tool-Augmented Assistants

GeoGPT extended a general-purpose language model with a set of tools for spatial queries and analysis, and demonstrated end-to-end GIS workflows driven by natural language (28). Subsequent work along the same line developed a fine-tuned variant operating GIS tools from annotated instruction–solution data (23), and a smaller model adapted to GIS question answering from domain corpora (30). The characteristic of this family is that grounding is achieved by *tool provisioning* — the agent is given a catalogue and asked to select — rather than by verification of the artifact produced.

A parallel line approached the problem from the generation side, training a language model specifically to emit tool-use chains for geospatial analysis in a simulated environment

(29), and to generate executable PyQGIS code (12). This line makes the target unambiguous: the unit of output is a script, and the evaluation is therefore about code. It also illustrates the dependency of the whole line on a fine-tuning corpus that, in the published instances, is proprietary in its preparation and expensive to reproduce — a property that motivates a retrieval-based alternative requiring no training data.

2.2.3 Benchmarking and Evaluation Practice

Evaluation of these systems has matured faster than the systems themselves, and the benchmark literature is arguably the most useful contribution of the recent period to anyone building such a tool.

A dedicated evaluation of code language models on geospatial code generation found that performance varies sharply by task framing, and that models plausible under loose evaluation can fail under strict signature-level checking (6). This is the finding that motivates the design of the benchmark used in this thesis, whose pass criteria are deliberately strict and whose verdict depends on the exact interface rather than on plausibility.

GeoLLM-Engine constructed an environment of more than a million queries over long-horizon geospatial tasks, and reported success rates near 77% for a commercial agent (21). Its contribution is the recognition that single-task, template-based benchmarks are unrepresentative: real analytical requests are long-horizon and require correct tool sequences. GeoAnalystBench took the complementary direction, assembling a benchmark of spatial analysis workflows with human-in-the-loop annotation to construct reference solutions (27).

The gap these benchmarks expose is the one this thesis occupies. Benchmarks exist; offline, reproducible generation systems that can be evaluated against them with static guarantees do not.

2.2.4 Multi-Agent and Hybrid Architectures

A more recent line combines chain-of-thought reasoning, retrieval and specialised collaborating agents, with fine-tuning to improve spatial task comprehension and tool selection (13). The direction is promising and the reported results encouraging, but the architecture assumes a hosted multi-agent service, which places it outside the offline constraint that governs this work.

Table 2.1 summarises the position of the reviewed systems along the dimensions that distinguish them for the present purpose.

2.3 Operational Assistants and Their Design Choices

Two operational systems illustrate the design space most directly: GeoPilot, a QGIS plugin, and GeoAgent, a library with a QGIS front-end. Neither is an integration target for this thesis, but their choices clarify what a benchmarkable, offline-first system must do differently.

Table 2.1: Comparison of reviewed natural-language-to-GIS systems. "Grounding" denotes the mechanism by which the model is constrained to the real API; "Verification" denotes whether the artifact is checked before or after generation.

System	Platform	Grounding	Execution	Verification
GeoGPT (28)	Platform-agnostic	Tool catalogue	Agent-internal	Tool-level
Autonomous GIS (11)	Platform-agnostic	Task decomposition	Generated + executed	None reported
GeoTool-GPT (23)	Commercial GIS API	Fine-tuning	Tool use via LLM	None
ChatGeoAI (12)	QGIS / PyQGIS	Fine-tuning + RAG	Script generation	None
GeoLLM-Engine (21)	Earth observation	Tool catalogue	Engine-internal	Model-checkers
GeoPilot	QGIS	Project context (live layers, CRS, fields)	`exec()` in QGIS	Regex guardlist, user confirmation
GeoAgent	QGIS, leafmap, anymap	Structured tool registry	Tool calls + gated script fallback	Confirmation hooks, metadata flags
GrassAgent (this work)	GRASS	Retrieval over documentation-derived schema	Headless subprocess, isolated mapset	Static AST + schema; runtime classification

GeoPilot. GeoPilot (5) lets a user drive a live QGIS session from a chat panel. It gathers the project's layers, coordinate reference systems, fields and available Processing algorithms, prompts a model with that state, parses fenced code blocks, and executes the result with `exec()` in a namespace containing QGIS objects. By default the user confirms the code before it runs; an optional auto-execute mode bypasses the confirmation, and a regular-expression guard blocks a short list of dangerous constructs (`os.system`, `subprocess`, `shutil.rmtree`, `eval`). Model access is broad and provider-agnostic, including a local Ollama backend.

Its grounding mechanism is *runtime project state*: the model is told what exists in the project rather than what the API permits. That is well suited to interactive work, where the user's layers are the usual source of error, but it leaves interface correctness entirely to the model. There is no signature-level check, and a code error surfaces as a traceback for the user to repair. Execution is unsandboxed at the Python level, mitigated only by confirmation, and the plugin offers no headless, reproducible evaluation path.

GeoAgent. GeoAgent (24) is a shared agent layer, built on the Strands Agents framework, that exposes geospatial Python functionality to a model through a structured tool registry. Its QGIS interface is distributed as the OpenGeoAgent plugin; the same library also binds to `leafmap`, `anymap`, SpatioTemporal Asset Catalogues and NASA Earthdata. Four abstractions organise it: a `GeoAgent` facade, `GeoAgentConfig` for provider and sampling settings, `GeoAgentContext` for live runtime objects, and a `@geo_tool` decorator that turns a typed function into a tool with its signature and docstring. Tools are grouped into inspection, navigation, layer management, data loading, selection and processing, and a confirmation-gated PyQGIS script fallback covers tasks without a dedicated tool. Tools carry risk metadata

such as `requires_confirmation`, `destructive` and `long_running`, which hooks intercept before execution.

Two choices matter here. First, *structural grounding*: the tool surface is declared statically, with typed signatures and docstrings, which is exactly the machine-checkable interface representation that deterministic verification needs. Second, *risk metadata as a first-class property*, which generalises the question of what generated code is permitted to do. The limitations are the mirror image: the integration is inherently interactive (a bound map, a QGIS interface, a dockable panel), it documents no headless evaluation mode, and a task outside the hand-curated tool surface must fall back on unverified generated code.

Other implementations. A survey of the QGIS plugin repository shows that the pattern is widespread. Several plugins bundle catalogues of tool and API definitions, retrieve from them before execution, and archive successful invocations; others advertise very large tool surfaces backed by a commercial model, or delegate entirely to aggregator APIs. What distinguishes the two systems above is not membership in this family but the articulation of their design: a documented grounding mechanism, an explicit execution model, and an explicit safety posture.

Lessons for the present work. GrassAgent adopts structural grounding and risk metadata, but derives the interface from the official manual rather than from hand-written declarations, and replaces interactive confirmation with process isolation because its evaluation is unattended. The comparison is therefore complementary rather than competitive: the two systems optimise for a present user in a live session, while this work optimises for a reproducible batch guarantee.

Bibliography

[1] Tom B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[2] Mark Chen et al. Evaluating large language models trained on code, 2021.

[3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 758–759, 2009.

[4] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. Automatic software repair: A survey. *IEEE Transactions on Software Engineering*, 45(1):34–67, 2019. doi: 10.1109/TSE.2017.2755013.

[5] GeoPilot Team. GeoPilot: AI-geospatial analysis assistant for QGIS, 2026. URL `https://github.com/xingguangYan/GeoPilot`. QGIS plugin repository identifier 5495; version 1.3.7, June 2026; MIT licence; accessed 2026-10-13.

[6] Piotr Gramacki, Bruno Martins, and Piotr Szymański. Evaluation of code LLMs on geospatial code generation. In *Proceedings of the 7th ACM SIGSPATIAL International Workshop on AI for Geographic Knowledge Discovery (GeoAI '24)*, pages 54–62, 2024. doi: 10.1145/3687123.3698286.

[7] GRASS GIS Project. *GRASS GIS Reference Manual: The Python API (grass.tools)*. Open Source Geospatial Foundation, 2026. URL `https://grass.osgeo.org/grass-stable/manuals/`. Version 8.6; accessed 2026-10-13.

[8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

[9] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020. doi: 10.18653/v1/2020.emnlp-main.550.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian

Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[11] Zhenlong Li and Huan Ning. Autonomous GIS: The next-generation AI-powered GIS. *International Journal of Digital Earth*, 16(2):4668–4686, 2023. doi: 10.1080/17538947.2023.2278895.

[12] Ali Mansourian and Rachid Oucheikh. ChatGeoAI: Enabling geospatial analysis for public through natural language, with large language models. *ISPRS International Journal of Geo-Information*, 13(10):348, 2024. doi: 10.3390/ijgi13100348.

[13] Ali Mansourian and Rachid Oucheikh. Bridging natural language and GIS: A multi-agent framework for LLM-driven geospatial analysis. *International Journal of Digital Earth*, 2026. doi: 10.1080/17538947.2026.2633849.

[14] Markus Neteler, M. Hamish Bowman, Martin Landa, and Markus Metz. GRASS GIS: A multi-purpose open source GIS. *Environmental Modelling & Software*, 31:124–130, 2012. doi: 10.1016/j.envsoft.2011.11.014.

[15] Python Software Foundation. *ast — Abstract Syntax Trees*, 2026. URL `https://docs.python.org/3/library/ast.html`. Python documentation; accessed 2026-10-13.

[16] QGIS Project. *PyQGIS Developer Cookbook*. QGIS.org, 2026. URL `https://docs.qgis.org/latest/en/docs/pyqgis_developer_cookbook/`. Accessed 2026-10-13.

[17] QGIS.org. QGIS geographic information system. Open Source Geospatial Foundation, 2026. URL `https://qgis.org`. Accessed 2026-10-13.

[18] Qwen Team. Qwen2.5-Coder technical report, 2024.

[19] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

[20] Noah Shinn, Federico Cassano, Ashish Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

[21] Simranjit Singh, Michael Fore, and Dimitrios Stamoulis. GeoLLM-Engine: A realistic environment for building geospatial copilots. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 585–594, 2024. doi: 10.1109/CVPRW63382.2024.00063.

[22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[23] Cheng Wei, Yifan Zhang, et al. GeoTool-GPT: A trainable method for facilitating large language models to master GIS tools. *International Journal of Geographical Information Science*, 39(4):707–731, 2024. doi: 10.1080/13658816.2024.2438937.

[24] Qiusheng Wu. GeoAgent: A multimodal AI agent for geospatial data analysis and interactive visualization, 2026. URL `https://github.com/opengeos/GeoAgent`. Distributed as the OpenGeoAgent QGIS plugin, repository identifier 5094, version 1.9.0, August 2026; MIT licence; accessed 2026-10-13.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izaak Shao, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2210.03629.

[26] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Yongji Wang, and Jian-Guang Lou. Large language models meet NL2Code: A survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7443–7464, 2023.

[27] Qianheng Zhang, Song Gao, Chen Wei, Yibo Zhao, Ying Nie, Ziru Chen, Shijie Chen, Yu Su, and Huan Sun. GeoAnalystBench: A GeoAI benchmark for assessing large language models for spatial analysis workflow and code generation. *Transactions in GIS*, 29(7):e70135, 2025. doi: 10.1111/tgis.70135.

[28] Yifan Zhang, Cheng Wei, Shangyou Wu, Zhengting He, and Wenhao Yu. GeoGPT: Understanding and processing geospatial tasks through an autonomous GPT, 2023. Published as "GeoGPT: An assistant for understanding and processing geospatial tasks", International Journal of Applied Earth Observation and Geoinformation, 2024, doi:10.1016/j.jag.2024.103976.

[29] Yifan Zhang, Jingxuan Li, Zhiyun Wang, Zhengting He, Qingfeng Guan, Jianfeng Lin, and Wenhao Yu. Geospatial large language model trained with a simulated environment for generating tool-use chains autonomously. *International Journal of Applied Earth Observation and Geoinformation*, 136:104312, 2024. doi: 10.1016/j.jag.2024.104312.

[30] Yifan Zhang, Zhiyun Wang, Zhengting He, Jingxuan Li, Gengchen Mai, Jianfeng Lin, Cheng Wei, and Wenhao Yu. BB-GeoGPT: A framework for learning a large language model for geographic information science. *Information Processing & Management*, 61(5): 103808, 2024. doi: 10.1016/j.ipm.2024.103808.